

Laboratorio 04: Clasificadores de ensamble

Asignatura: *Machine Learning* (2026-01)

Profesor: Dr. Juan Bekios Calfa

1st Pablo López

Escuela de Ingeniería

Universidad Católica del Norte

Ingeniería Civil en Computación e Informática

pablo.lopez01@alumnos.ucn.cl

2nd Ignacio García

Escuela de Ingeniería

Universidad Católica del Norte

Ingeniería Civil en Computación e Informática

ignacio.garcia02@alumnos.ucn.cl

3rd Eros Cortés

Escuela de Ingeniería

Universidad Católica del Norte

Ingeniería Civil en Computación e Informática

eros.cortes@alumnos.ucn.cl

Resumen—Este informe presenta el desarrollo del Laboratorio 04 de Machine Learning, centrado en la comparación de clasificadores de ensamble aplicados a un dataset SPSS con 15 atributos y seis objetivos de clasificación. Se implementaron y evaluaron Bagging, AdaBoost, Stacking y Gradient Boosting usando validación cruzada anidada, con *GridSearchCV* y *RandomizedSearchCV* para la selección de hiperparámetros. La métrica principal fue F1 macro, debido al desbalance observado en las clases. Los mejores resultados variaron según el objetivo: Stacking obtuvo el mayor F1 macro en GDS, Gradient Boosting destacó en GDS_R1 y GDS_R2, AdaBoost en GDS_R3, y Bagging en GDS_R4 y GDS_R5. Los resultados muestran que no existe un ensamble dominante para todos los objetivos y que la evaluación debe considerar tanto desempeño promedio como estabilidad y soporte de clases.

El objetivo general es analizar el comportamiento de clasificadores de ensamble sobre los seis objetivos del dataset. Como objetivos específicos, se busca configurar hiperparámetros desde un archivo YAML, ejecutar búsquedas exhaustivas y aleatorias, reportar métricas macro adecuadas para clases desbalanceadas, revisar matrices de confusión y discutir las limitaciones metodológicas derivadas del bajo soporte en algunas clases.

El desarrollo se implementó en Python utilizando *scikit-learn*, *pandas*, *numpy*, *scipy*, *pyreadstat* y *PyYAML*. La ejecución se organizó mediante módulos separados para carga de datos, definición de modelos, evaluación y generación de reportes.

I. INTRODUCCIÓN

El laboratorio aborda un problema de clasificación supervisada sobre un conjunto de datos originalmente almacenado en formato SPSS. El dataset contiene 1119 registros, 15 atributos de entrada y seis objetivos de clasificación: GDS, GDS_R1, GDS_R2, GDS_R3, GDS_R4 y GDS_R5. Cada objetivo representa una agrupación distinta de clases, por lo que el mismo conjunto de atributos se evalúa bajo escenarios de clasificación con distinta dificultad y distinto nivel de desbalance.

La motivación principal es comparar estrategias de ensamble bajo un protocolo experimental reproducible. Los ensambles combinan varios modelos individuales para mejorar la capacidad predictiva, reducir varianza o corregir errores secuencialmente. En este laboratorio se evalúan cuatro enfoques: Bagging, AdaBoost, Stacking y Gradient Boosting. La comparación se realiza con validación cruzada anidada para separar la selección de hiperparámetros de la estimación final del rendimiento.

II. MARCO TEÓRICO

Un clasificador de ensamble combina varios modelos base para producir una predicción final. La hipótesis general es que una combinación de clasificadores puede obtener un rendimiento más robusto que un único modelo individual, especialmente cuando los errores de los modelos base no son idénticos.

Bagging entrena múltiples clasificadores sobre muestras bootstrap del conjunto de entrenamiento. En este laboratorio se usa *BaggingClassifier* con árboles de decisión como estimadores base. Su principal ventaja es la reducción de varianza, ya que promedia o vota entre modelos entrenados sobre subconjuntos distintos de datos.

AdaBoost construye un ensamble secuencial de clasificadores débiles. Después de cada iteración, aumenta el peso de los ejemplos mal clasificados, obligando al siguiente estimador a enfocarse en casos difíciles. En este laboratorio se emplean árboles de decisión poco profundos como clasificadores base.

Stacking combina modelos base mediante un metaclasificador. Los modelos base generan predicciones o proba-

bilidades que se usan como nuevas características para entrenar un clasificador final. La implementación usada incluye árbol de decisión, K-NN escalado y regresión logística escalada, con regresión logística como meta-modelo.

Gradient Boosting también es un método secuencial, pero cada nuevo árbol intenta corregir los errores residuales del ensamble acumulado. Los hiperparámetros relevantes incluyen número de estimadores, tasa de aprendizaje, profundidad de los árboles y proporción de muestras usadas en cada etapa.

La métrica principal fue F1 macro. Esta métrica calcula el F1 de cada clase y luego promedia todos los valores, asignando el mismo peso a cada clase. Esto es apropiado cuando existe desbalance, ya que evita que una clase mayoritaria domine la evaluación. También se reportaron *balanced accuracy*, precisión macro, recall macro, estabilidad e ICN. La validación cruzada anidada se utilizó para evitar una estimación optimista: el ciclo interno selecciona hiperparámetros y el ciclo externo estima el rendimiento final.

III. METODOLOGÍA

El dataset contiene 1119 observaciones y 15 atributos de entrada. Los atributos corresponden a variables codificadas como valores numéricos, y los objetivos corresponden a distintas agrupaciones de clases. La Tabla I resume el soporte por objetivo.

Objetivo	Clases y soporte	n_{min}
GDS	1:149, 2:500, 3:298, 4:108, 5:42, 6:20, 7:2	2
GDS_R1	1:947, 2:150, 3:22	22
GDS_R2	1:649, 2:298, 3:172	172
GDS_R3	1:947, 3:172	172
GDS_R4	1:149, 2:906, 3:64	64
GDS_R5	1:149, 2:798, 3:172	149

Tabla I: Distribución de clases para los seis objetivos.

La carga de datos se realizó desde el archivo `datasets/15 atributos R0-R5.sav`. Se seleccionaron las 15 columnas de atributos definidas en el código y se transformaron a tipo flotante. Cada objetivo se convirtió a entero para ser usado por los clasificadores. No se aplicó balanceo artificial ni modificación del dataset, ya que el foco del laboratorio fue comparar ensambles bajo un protocolo común.

Los modelos se organizaron en `src/models.py`. La evaluación se implementó en `src/evaluation.py`, donde se calcula el número de folds externos como:

$$k_{outer} = \min(5, n_{min}) \quad (1)$$

Esta regla evita solicitar más particiones estratificadas que ejemplos disponibles en la clase minoritaria. En el caso de GDS, el soporte mínimo es 2; por tanto, el ciclo externo queda limitado a dos folds. Además, en algunos entrenamientos externos una clase queda con solo un ejemplo, por lo que el ciclo interno utiliza *KFold* no estratificado y registra una advertencia metodológica.

Los hiperparámetros se configuraron en `config/paths.yaml`. Se ejecutaron dos experimentos:

`grid_all`, basado en *GridSearchCV*, y `random_all`, basado en *RandomizedSearchCV*. Ambos usaron F1 macro como métrica de selección. Las semillas de aleatoriedad se centralizaron para controlar particiones, modelos y búsquedas aleatorias.

IV. RESULTADOS

La Tabla II muestra el mejor resultado por objetivo considerando todos los modelos y ambos tipos de búsqueda. Los valores corresponden al promedio de F1 macro sobre los folds externos.

Objetivo	Mejor modelo	Búsqueda	F1 macro
GDS	Stacking	grid	0.362
GDS_R1	Gradient Boosting	grid	0.729
GDS_R2	Gradient Boosting	grid	0.669
GDS_R3	AdaBoost	random	0.800
GDS_R4	Bagging	grid	0.615
GDS_R5	Bagging	random	0.577

Tabla II: Mejor F1 macro por objetivo.

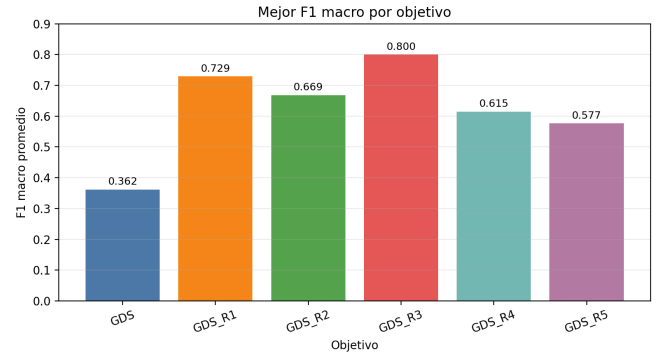


Figura 1: Mejor F1 macro promedio por objetivo.

La Tabla III resume métricas adicionales para los mejores modelos. Se observa que los objetivos GDS_R3 y GDS_R1 alcanzan los mayores F1 macro, mientras que GDS es el caso más débil debido al fuerte desbalance y al soporte mínimo de solo dos ejemplos en una clase.

Para analizar errores específicos se revisaron matrices de confusión. En GDS_R3, el mejor modelo fue AdaBoost con búsqueda aleatoria. La matriz de confusión agregada se muestra en la Tabla IV. El modelo clasificó correctamente 894 ejemplos de la clase 1 y 111 ejemplos de la clase 3. El F1 por clase fue 0.940 para la clase 1 y 0.661 para la clase 3, evidenciando el efecto del desbalance.

En GDS_R5, el mejor resultado fue obtenido por Bagging con búsqueda aleatoria. La Tabla V muestra que la clase 2, mayoritaria, concentra la mayor cantidad de aciertos, mientras que la clase 1 tiene menor F1 por clase. Los F1 por clase fueron 0.365, 0.715 y 0.650 para las clases 1, 2 y 3, respectivamente.

V. DISCUSIÓN

Los resultados muestran que no existe un único modelo ganador para todos los objetivos. Stacking fue el mejor en

Objetivo	Modelo	F1 macro	Bal. Acc.	Prec. macro	Recall macro	ICN
GDS	Stacking	0.362	0.470	0.363	0.470	0.907
GDS_R1	Gradient Boosting	0.729	0.718	0.762	0.718	0.658
GDS_R2	Gradient Boosting	0.669	0.649	0.703	0.649	0.552
GDS_R3	AdaBoost	0.800	0.795	0.808	0.795	0.692
GDS_R4	Bagging	0.615	0.649	0.621	0.649	0.854
GDS_R5	Bagging	0.577	0.629	0.568	0.629	0.853

Tabla III: Métricas de los mejores modelos por objetivo.

Real / Pred.	1	3
1	894	53
3	61	111

Tabla IV: Matriz de confusión para GDS_R3 con AdaBoost random.

Real / Pred.	1	2	3
1	82	65	2
2	215	508	75
3	3	49	120

Tabla V: Matriz de confusión para GDS_R5 con Bagging random.

GDS, Gradient Boosting obtuvo los mejores resultados en GDS_R1 y GDS_R2, AdaBoost destacó en GDS_R3, y Bagging fue superior en GDS_R4 y GDS_R5. Esto confirma que el comportamiento de los ensambles depende de la estructura del objetivo, el nivel de desbalance y la forma en que se agrupan las clases.

El caso GDS debe interpretarse con especial cautela. Aunque Stacking obtuvo el mayor F1 macro para este objetivo, la clase menos frecuente tiene solo dos ejemplos. Esto limita la validación externa a dos folds y obliga al ciclo interno a usar *KFold* no estratificado cuando una clase queda con un solo ejemplo en entrenamiento. Por tanto, el valor de F1 macro en GDS es menos estable que en los demás objetivos.

En GDS_R3 se obtuvo el mejor F1 macro global, superando levemente 0.80. Sin embargo, las métricas por clase muestran que el rendimiento no es homogéneo: la clase mayoritaria alcanza F1 de 0.940, mientras que la clase minoritaria alcanza 0.661. Esto evidencia la importancia de usar F1 macro y no solo exactitud global. En GDS_R5, Bagging logra buen comportamiento relativo, pero la matriz de confusión muestra confusión importante entre las clases 1 y 2.

La comparación entre *GridSearchCV* y *RandomizedSearchCV* también resulta relevante. Algunos objetivos favorecieron la búsqueda exhaustiva y otros la búsqueda aleatoria. *RandomizedSearchCV* permitió explorar rangos más amplios sin evaluar todas las combinaciones posibles, mientras que *GridSearchCV* fue útil en espacios pequeños y controlados.¹ La configuración desde YAML facilitó modificar los hiperparámetros sin alterar la lógica del código.

VI. CONCLUSIONES

El laboratorio permitió comparar cuatro estrategias de ensamble bajo un protocolo reproducible y defendible. La principal conclusión es que no existe un ganador universal:

cada objetivo favoreció un modelo distinto. Esto refuerza la necesidad de evaluar múltiples modelos y no seleccionar una técnica solo por su desempeño en un único objetivo.

El uso de F1 macro fue adecuado debido al desbalance de clases. Métricas como accuracy habrían sido menos informativas, especialmente en objetivos donde una clase concentra gran parte de los registros. La validación cruzada anidada permitió separar selección de hiperparámetros y estimación del desempeño, reduciendo el riesgo de optimismo experimental.

La principal limitación fue el bajo soporte de algunas clases, especialmente en GDS. Esta condición no invalida el experimento, pero obliga a interpretar sus resultados con cautela. Como trabajo futuro se podría comparar el efecto de estrategias de balanceo, siempre que el objetivo metodológico lo permita, o explorar criterios de selección que penalicen de forma más explícita la inestabilidad entre folds.

VII. CÓDIGO Y REPRODUCIBILIDAD

El proyecto se organizó de forma modular. El archivo `main.py` orquesta la ejecución completa. El módulo `src/data_loader.py` carga el archivo SPSS y prepara matrices de atributos y objetivos. El módulo `src/models.py` define los clasificadores de ensamble. El módulo `src/evaluation.py` implementa validación cruzada anidada, búsquedas de hiperparámetros y cálculo de métricas. Finalmente, `src/reports.py` genera archivos CSV, JSON, LaTeX, PDF, matrices de confusión y métricas por clase.

La configuración experimental se encuentra en `config/paths.yaml`. En este archivo se definen rutas, objetivos, experimentos, semillas, métrica de selección, número máximo de folds e hiperparámetros para búsqueda grid y random. Esta separación mejora la reproducibilidad porque permite modificar el experimento sin alterar el código fuente.

La ejecución completa se realiza con el comando mostrado en el Código 1. También se pueden ejecutar subconjuntos de objetivos, modelos o experimentos mediante argumentos de línea de comandos.

```
python main.py
```

Código 1: Ejecución completa del laboratorio

Los resultados principales se almacenaron en `outputs/tables/resumen_resultados.csv`. Las advertencias metodológicas quedaron en `outputs/advertencias.txt`; la advertencia principal corresponde a GDS por soporte insuficiente en el ciclo

interno. Además, algunas combinaciones aleatorias de AdaBoost fueron descartadas por *scikit-learn* al producir ajustes no válidos, quedando registradas con puntaje no finito. La ejecución finalizó correctamente y generó tablas, matrices de confusión y métricas por clase.

APÉNDICE A ANEXOS

Los archivos generados por el laboratorio incluyen:

- `outputs/tables/resumen_resultados.csv`: resumen tabular de métricas por modelo, objetivo y experimento.
- `outputs/tables/resultados_detallados.json`: resultados completos de la evaluación.
- `outputs/confusion_matrices/`: matrices de confusión agregadas.
- `outputs/per_class/`: métricas por clase.
- `outputs/advertencias.txt`: advertencias metodológicas.

APÉNDICE B RÚBRICA DE AUTOEVALUACIÓN

Criterio	¿Cumplido?
Introducción clara	Sí
Marco teórico adecuado	Sí
Metodología detallada	Sí
Resultados completos y visuales	Sí
Discusión crítica	Sí
Conclusiones reflexivas	Sí
Calidad y estructura del código	Sí
Presentación clara y sin errores	Sí
Reproducibilidad	Sí
Aportes adicionales	Sí

Tabla VI: Tabla de autoevaluación.